Contents

1 Linear regression

Linear regression is an essential, basic tool in building models for prediction. It expresses our output variable y_i as a function of our predictor variables x_i. For example, in the example of predicting a student's exam result based on their time studied, a simple linear regression assumes the model:

$$\text{exam result} = \beta_0 + \beta_1 \times \text{time studied}$$

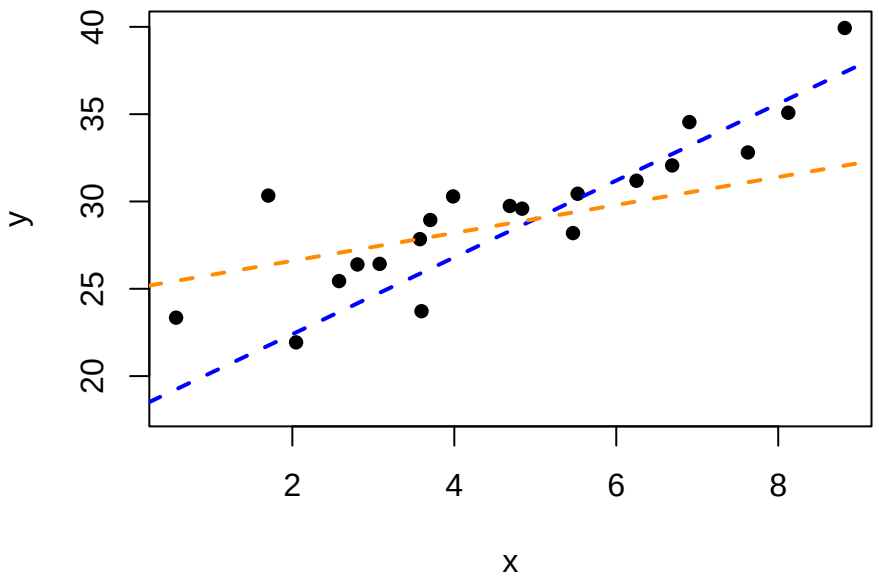
which is exactly of the form $y = mx + c$. In this model, we have two parameters to choose: β_0, the intercept term, and β_1, the coefficient (gradient) of the time studied.

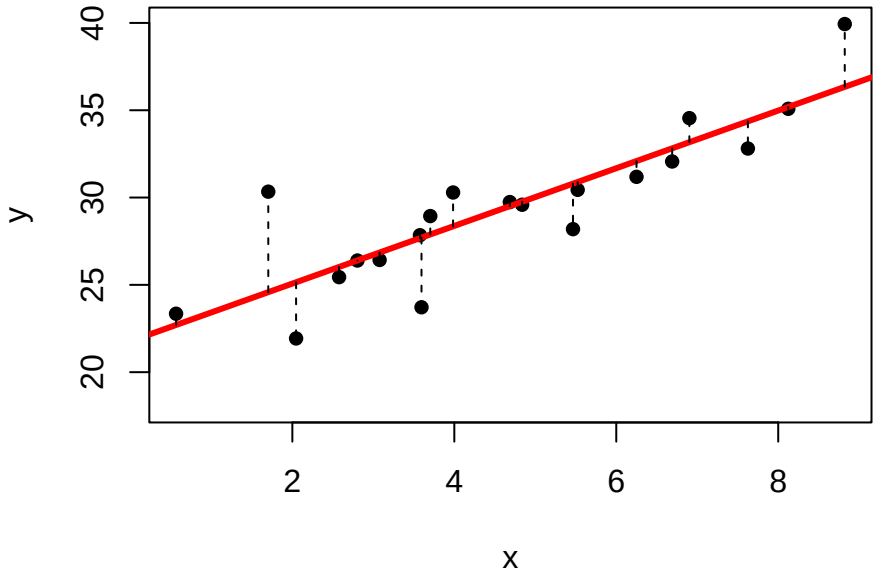
For example, say that after training our model, we find $\beta_0 = 50$ and $\beta_1 = 2$. This implies that when $\text{time studied} = 0$, the model predicts they will score 50 on their exam. Likewise, for every unit increase in time studied, their score will increase by 2.

Unlike kNN, we do not need to scale the data in linear regression, as there is no notion of 'distance' *between* predictors used in linear regression.

1.1 The 'line of best fit'

Linear regression is commonly called 'drawing the line of best fit'. The way in which a line drawn is 'best' is defined simply: given some dataset, we want to draw some line which minimises the distance from that line to the dataset. Specifically, we choose to minimise the **squared** distances. For example, in the below graph, the orange and blue lines are 'lines of *some* fit'. But the red one minimises the squared distance between the points and the line, and is the line of best fit.





2 Setup

In this lab, we will be looking at the same `Boston` dataset from the `MASS` package. Again, we will be predicting the `medv` value based on some predictors.

```
data("Boston", package="MASS")
```

This means that we will be expressing the `medv` as a function of all our predictors:

$$\text{medv} = \beta_0 + \beta_1 \times \text{crim} + \beta_2 \times \text{zn} + \cdots + \beta_{13} \times \text{lstat}$$

So, this linear regression model has 14 parameters, $\beta_0, \beta_1, \ldots, \beta_{13}$, where β_0 is the intercept and $\beta_1, \ldots, \beta_{13}$ are the coefficients (gradients) of each of the predictors. In the training process, we will aim to find the 'best' values for the parameters $\beta_0, \ldots, \beta_{13}$.

2.1 Train-test split

We will use the same train/test data as the kNN model, so that we can directly compare each model.

```
set.seed(100)
train.idx <- sample(nrow(Boston), nrow(Boston)*0.8)
train.data <- Boston[train.idx,]
test.data <- Boston[-train.idx,]
```

2.2 Training the model

To build and train a linear regression model, we will use the built-in `lm()` function. Here, we want to predict `medv` using all other features (denoted with the `.`).

```
m.slr <- lm(medv ~ ., data = train.data)
```

We can view the coefficients of $\beta_0, \ldots, \beta_{13}$ by looking at `m.slr`. For example, the β_0 term, the intercept, has a value of `36.48`. β_1, representing the coefficient of `crim`, has a value of `-0.127`.

```
m.slr
```

```
##
## Call:
## lm(formula = medv ~ ., data = train.data)
##
## Coefficients:
## (Intercept)         crim           zn        indus         chas           nox
##   36.485856    -0.127464     0.048885     0.038884     3.627401   -
12.727135
##          rm          age          dis          rad          tax      ptratio
##    3.509160    -0.006043    -1.452142     0.277477    -0.009174    -
0.967862
##       black        lstat
##    0.008838    -0.644685
```

To see more details about the model, we can call the `summary()` function on the model. While this gives us a lot of information, in this course, we will only look at a few parts of this summary:

1. the `Estimate` column;
2. the column with the stars `***`; and
3. the line containing `R-squared`.

```
summary(m.slr)
```

```
##
## Call:
## lm(formula = medv ~ ., data = train.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.6610  -2.6944  -0.3543   1.8265  24.3483
```

```
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  36.485856   5.590340   6.527 2.10e-10 ***
## crim         -0.127464   0.031240  -4.080 5.46e-05 ***
## zn            0.048885   0.013986   3.495 0.000528 ***
## indus         0.038884   0.063707   0.610 0.541980
## chas          3.627401   0.950492   3.816 0.000157 ***
## nox         -12.727135   3.918540  -3.248 0.001263 **
## rm            3.509160   0.467747   7.502 4.28e-13 ***
## age          -0.006043   0.014184  -0.426 0.670321
## dis          -1.452142   0.212117  -6.846 2.96e-11 ***
## rad           0.277477   0.065699   4.223 3.00e-05 ***
## tax          -0.009174   0.003767  -2.435 0.015327 *
## ptratio      -0.967862   0.135561  -7.140 4.61e-12 ***
## black         0.008838   0.002721   3.248 0.001263 **
## lstat        -0.644685   0.054764 -11.772  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.351 on 390 degrees of freedom
## Multiple R-squared:  0.7697, Adjusted R-squared:  0.762
## F-statistic: 100.3 on 13 and 390 DF,  p-value: < 2.2e-16
```

The `Estimate` column contains the same coefficients $\beta_0, \ldots, \beta_{13}$ we saw earlier. These coefficients form our model.

2.3 p-values

The column with the stars `***` are p-values and contains information on whether that predictor (or intercept) is significant in *predicting the outcome value*. For the purposes of this course, the more stars, the more significant the predictor is.

To give a quick explanation of it, suppose we have a coin. We want to test whether it is a fair coin, i.e. equal probability for heads or tails. Suppose we flip it 10 times, we would expect 5 heads and 5 tails. Any sequence of 10 flips deviating far from 5 heads and 5 tails is evidence against it being a fair coin.

The table below shows the p-values given the number of heads observed, and assuming we have a fair coin. If we observe 0 or 1 heads (correspondingly, 9 or 10 heads), that is strong **statistical evidence** that the coin is not a fair coin. If we observe 2 (or 8) heads, that is **some** statistical evidence that the coin is not fair. Anything between 3 to 7 heads does not pose strong statistical evidence that the coin is not fair - they could have happened purely by chance.

Generally, 5% = 0.05 is used as a threshold for 'statistically significant' results, so anything less than 0.05 is statistically significant. **Note that this does not mean it is causally associated! It is just a**

correlation in the data that we observe. A fair coin could also have generated 0 or 1 (9 or 10) head flips, it is just unlikely to do so.

```
##     0     1     2     3     4     5     6     7     8     9    10
## 0.002 0.021 0.109 0.344 0.754 1.000 0.754 0.344 0.109 0.021 0.002
```

p-values are also widely used in research, but there are valid criticisms of it (see p-value hacking).

Extending this to the linear regression results from earlier, we can see that all variables except `indus` and `age` have a significant p-value.

2.4 R-squared (R^2)

The R^2 value measures how close the data are to the trained (fitted) regression model. Here, our R^2 is 0.7697, or 76.97%, indicating that this fitted model can explain 76.97% of the variation in the outcome variable `medv`. The other 23.03% is 'not accounted for' in this model, and we can think of this being due to noise ('natural variation') or other predictors we have not measured.

The *adjusted* R^2 is an alternative unbiased measurement for R^2 that accounts for the difference in sample size and number of parameters of our model. The R^2 and adjusted R^2 values should be near to each other, unless the number of parameters in our model is high compared to the number of observations. In this case, we have 404 observations in the train data and 14 parameters in the model. If, for example, we increase our number of parameters to 202, with the same number of observations, the R^2 and adjusted R^2 values will differ greatly.

This R^2 value, however, cannot be used to justify the "goodness" of a regression model, as it only is relevant **to the training data**. The R^2 value of a model increases as more parameters are added to the model. A high R^2 value does not indicate that it will perform well in the real world, i.e. it may have a large RMSE on the test set.

3 Testing the model

We follow through with the same procedure for testing the model. We get a test RMSE of 6.278, which is worse than the kNN model's 5.714.

```
yhat <- predict(m.slr, test.data)
y <- test.data$medv
rmse.slr <- RMSE(yhat, y)
rmse.slr
```

```
## [1] 6.277736
```

```r
ggplot(cbind(yhat, y), aes(yhat, y)) + geom_point() +
  geom_abline(slope=1, intercept=0, col="red") +
  labs(title="Linear regression model performance on test set",
       x="Predicted", y="Actual")
```

4 Analysing the model

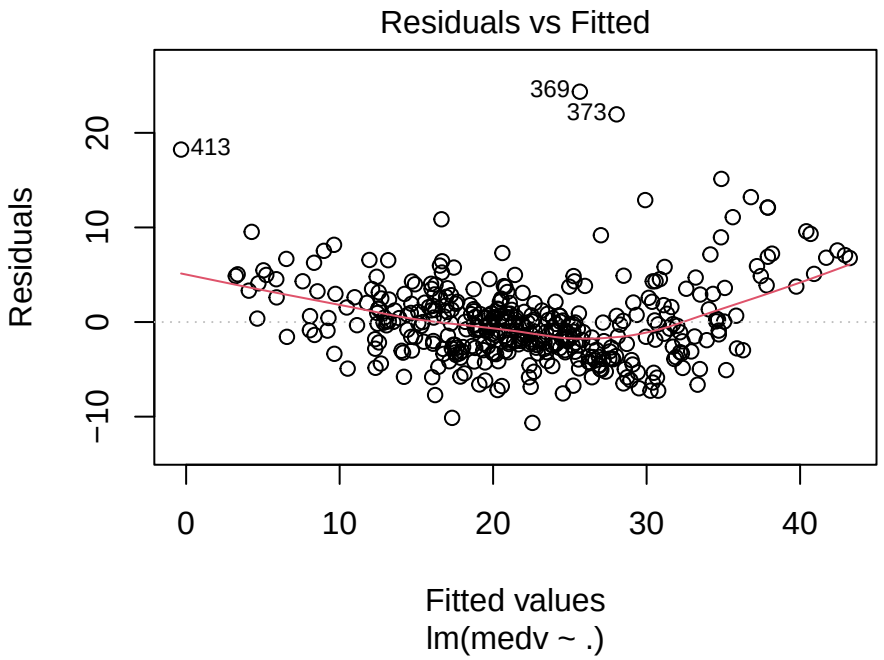
To analyse the model, we can look at the residuals $\hat{y}_i - y_i$ of the model against the training data. Plotting the model object `m.slr` will give us some diagnostic charts of the model. For this course, we will only focus on the first plot of 'Residuals vs Fitted'.

The `par()` function changes settings when plotting with the built-in `plot()` function. The settings changed in `par()` will remain in place until changed again, or R is restarted. The `mfrow` setting takes a vector of two numbers, and defines how many plots we want at the same time. In this case, `c(2,2)` is two rows and two columns.

```r
par(mfrow=c(2,2))
plot(m.slr)
par(mfrow=c(1,1))   # change back settings to default
```

We can also get only the plot of the residuals by using the parameter `which=1`.

```r
plot(m.slr, which=1)
```



This plot shows all the predicted (fitted) values against its residuals. In general, a good linear regression model will have its residuals symmetrically distributed above and under the horizontal line at 0, with no clear patterns. The red line is a 'best fit' line, and in this case indicates a quadratic-esque pattern in the residuals.

We can improve the model to account for this quadratic pattern in the residuals.

4.1 Improving the model

While the tutorial notes suggest removing the 'outliers' indicated in the first residual plot, I do not see a strong reason to remove them apart from making your model look nicer.

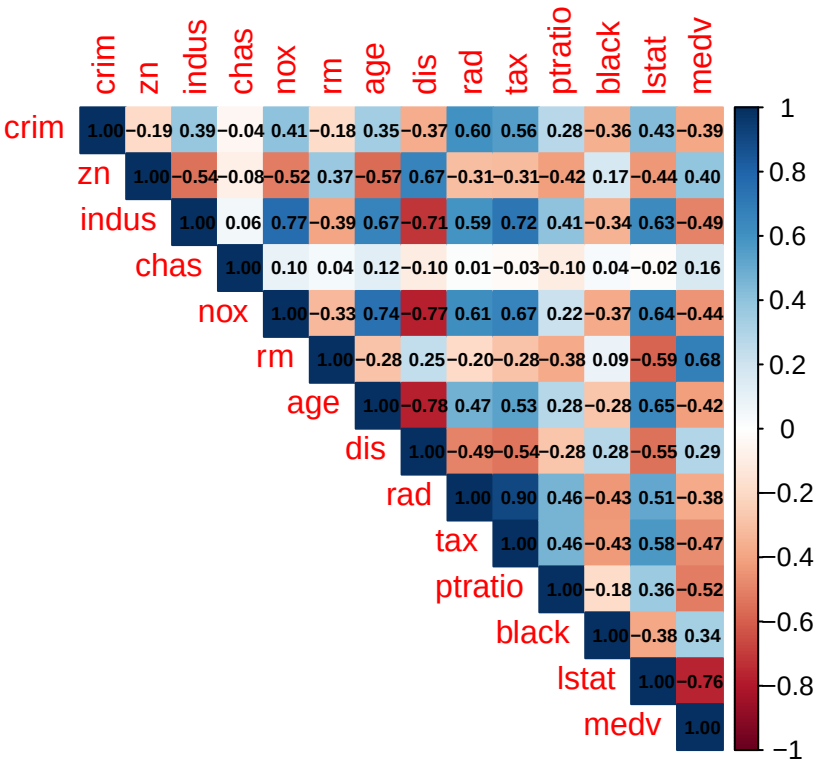
Further, in the tutorial notes, the datasets for the second model are completely different from the first model, so the two models in the tutorial notes are not comparable. Instead, I will look at only adding quadratic terms here.

A model can always be 'improved' to fit the training data better by adding more parameters. So, we must consider which parameters to add. The residual graph, having a quadratic shape, suggests quadratic terms may help. There are 13 possible predictors we can add quadratic terms to, and to help in choosing which, the tutorial notes look at the correlation coefficient between every predictor and `medv`.

To see the correlation coefficient between every variable in our dataset, we will use the `corrplot()` function from the `corrplot` library. Install the library as necessary. `cor()` calculates the correlation

coefficient between every variable, and `corrplot()` simply shows its results graphically. Some parameters are changed to make the plot more interpretable.

```
library(corrplot)
corrplot(cor(train.data),
         type="upper",            # show upper half only
         method="color",          # fill squares
         addCoef.col = "black",   # add the correlation coefficient
         number.cex = 0.6)        # resize the correlation coefficient text
```



Predictors with a high correlation coefficient with the outcome variable `medv` are chosen, `rm`, `ptratio`, and `lstat`. To add quadratic terms, we add them to the right of `medv ~ .`, but must surround each term in `I()` for it to be interpreted by R correctly. The R^2 value increases from 0.7697 to 0.8557.

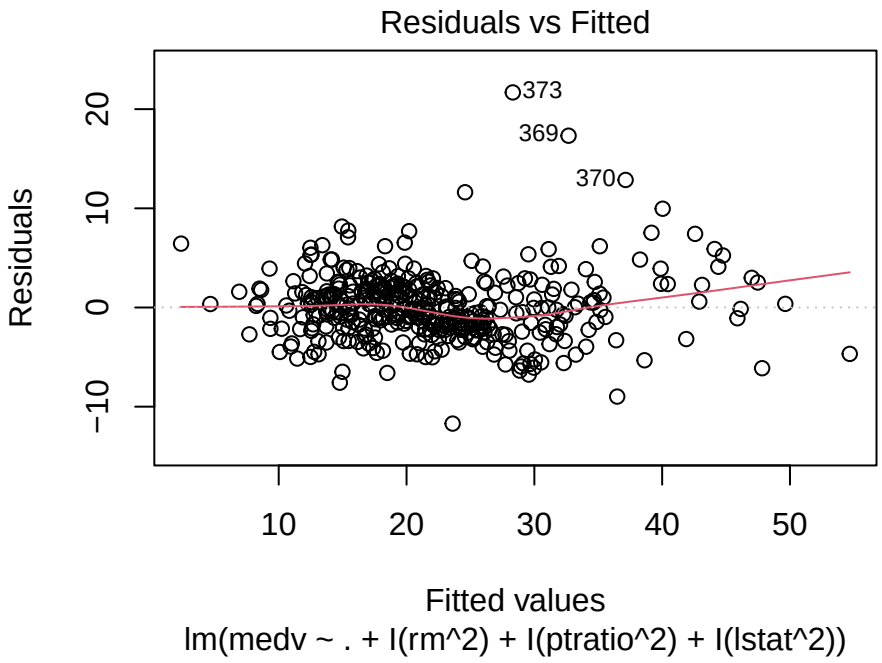
```
m.quadratic <- lm(medv ~ . + I(rm^2) + I(ptratio^2) + I(lstat^2), data =
↪  train.data)
summary(m.quadratic)
```

```
##
## Call:
## lm(formula = medv ~ . + I(rm^2) + I(ptratio^2) + I(lstat^2),
##     data = train.data)
##
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -11.7066  -2.0333  -0.1843   1.7786  21.6864
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)  170.598415  16.743161  10.189  < 2e-16 ***
## crim          -0.151474   0.024967  -6.067 3.11e-09 ***
## zn             0.015612   0.011639   1.341  0.18061
## indus          0.096728   0.050852   1.902  0.05790 .
## chas           4.411651   0.766206   5.758 1.74e-08 ***
## nox          -15.198846   3.243367  -4.686 3.86e-06 ***
## rm           -27.818270   3.159947  -8.803  < 2e-16 ***
## age            0.002382   0.011703   0.204  0.83885
## dis           -1.003445   0.171420  -5.854 1.03e-08 ***
## rad            0.254557   0.053224   4.783 2.46e-06 ***
## tax           -0.008581   0.003014  -2.847  0.00464 **
## ptratio       -4.686094   1.557008  -3.010  0.00279 **
## black          0.006550   0.002174   3.013  0.00276 **
## lstat         -1.396404   0.130070 -10.736  < 2e-16 ***
## I(rm^2)        2.426541   0.250688   9.680  < 2e-16 ***
## I(ptratio^2)   0.113225   0.043817   2.584  0.01013 *
## I(lstat^2)     0.022097   0.003556   6.214 1.33e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.462 on 387 degrees of freedom
## Multiple R-squared:  0.8554, Adjusted R-squared:  0.8494
## F-statistic:   143 on 16 and 387 DF,  p-value: < 2.2e-16
```

Examining the residual plot of this model, we also see the quadratic pattern is reduced. This model with quadratic terms, when compared to the previous model, is significantly better, in terms of the residual plot. Note that this does not imply anything about how it will perform in the real world - we still must evaluate the model with the test set.

```
plot(m.quadratic, which=1)
```



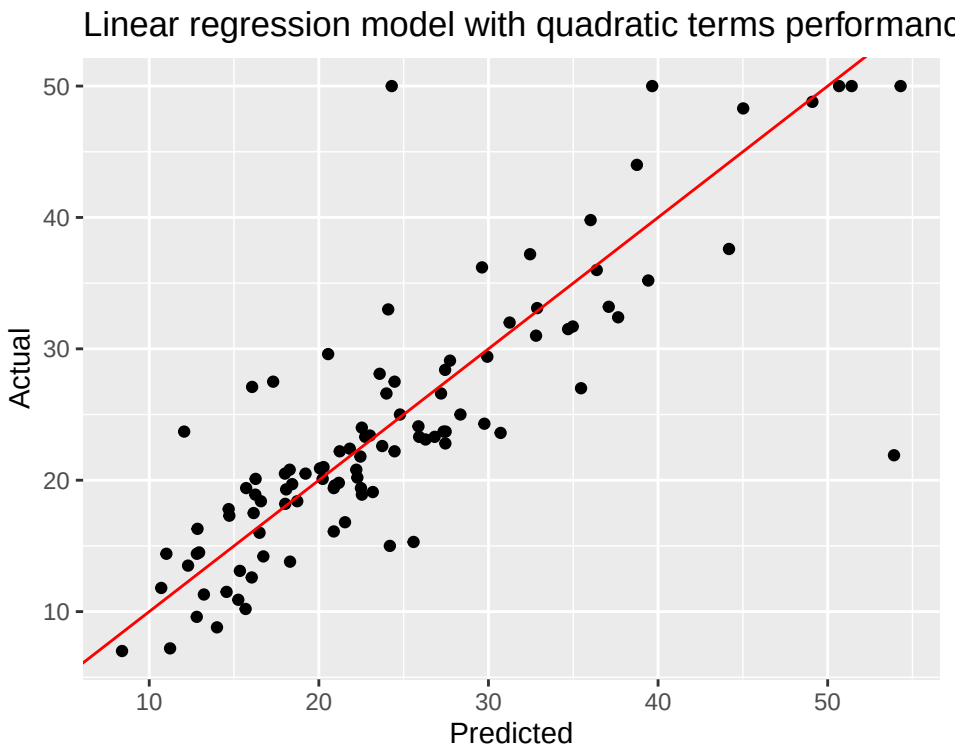
4.2 Testing the improved model

With the quadratic terms, the model's test set RMSE improves 16% from 6.278 to 5.789. While not as stellar as the tutorial note's improvement to 2.890, the difference in the tutorial notes is due to the different datasets used between each model.

```
yhat <- predict(m.quadratic, test.data)
rmse.quadratic <- RMSE(yhat, y)
cbind(rmse.slr, rmse.quadratic)
```

```
##      rmse.slr rmse.quadratic
## [1,] 6.277736       5.801981
```

```
ggplot(cbind(yhat, y), aes(yhat, y)) + geom_point() +
  geom_abline(slope=1, intercept=0, col="red") +
  labs(title="Linear regression model with quadratic terms performance on
  ↪  test set",
       x="Predicted", y="Actual")
```



5 Transformations

Sometimes, the relations between predictors and outcome is not linear or quadratic, and using some transformation on the predictors (or outcome) can yield a better model.

```
ggplot(train.data, aes(lstat, medv)) + geom_point() +
  geom_smooth(method = lm, formula = y ~ log(x))
ggplot(train.data, aes(log(lstat), medv)) + geom_point() +
  geom_smooth(method = lm, formula = y ~ x)
```

Plotting `lstat` against `medv`, we can see that there is a (negative) logarithmic trend. Thus, plotting `log(lstat)` against `medv` will have a linear trend. We can thus model `medv` as a function of ‘log(lstat), i.e.

$$\text{medv} = \beta_0 + \beta_1 \times \log(\text{lstat})$$

In this model, there are only two parameters, which are again determined through the training process. We will use the same training data as the other two so that we can compare all the models. This simple 2-parameter model achieves a test RMSE of 6.634, which is impressive for such a small model.

```
m.loglstat <- lm(medv ~ log(lstat), data = train.data)
summary(m.loglstat)
```

```
##
## Call:
## lm(formula = medv ~ log(lstat), data = train.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -14.1364  -3.2054  -0.4874   2.2294  25.4383
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  51.5164     1.0067   51.17   <2e-16 ***
## log(lstat)  -12.3430     0.4105  -30.07   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.954 on 402 degrees of freedom
## Multiple R-squared:  0.6923, Adjusted R-squared:  0.6915
## F-statistic: 904.3 on 1 and 402 DF,  p-value: < 2.2e-16
```

```
yhat <- predict(m.loglstat, test.data)
rmse.loglstat <- RMSE(yhat, y)
rmse.all <- rbind(rmse.slr, rmse.quadratic, rmse.loglstat)
n.params.all <- rbind(14, 17, 2)
data.frame(rmse = rmse.all, n.params = n.params.all,
           row.names = c("slr", "quadratic", "loglstat"))
```

```
## # A tibble: 3 x 2
##    rmse n.params
##   <dbl>    <dbl>
## 1  6.28       14
## 2  5.80       17
## 3  6.63        2
```

6 Extra: limitations

Unlike kNN, linear regression imposes a relationship between the outcome variable and the predictors. While linear regression can give a *reasonable* prediction with extrapolation of this relationship (unlike kNN), this relationship usually does not extrapolate to infinity. For example, a medication might show no effect below 50mg, optimal effect at 50–100mg, and toxicity above 100mg. A simple linear regression would incorrectly model the medication's efficacy as a straight line, missing the critical thresholds.

We may use alternate models to model this behaviour (for example, we used quadratic terms and logarithmic terms), but this is outside of the scope of this course.

Outliers can also affect the model produced by linear regression. This is due to how the model is fitted, which is by minimising the squared distance from the line to the dataset. Reducing the distance to the outlier point at the cost of every other point reduces this total distance, even if it is detrimental to the overall model.

One last issue that occurs is when predictors are co-linear (collinear, correlated). Having predictors which are correlated to each other leads to being unable to attribute the effect of one predictor variable to another. If, for example, we have two predictors x and y and outcome z, and one predictor can be explicitly expressed in terms of the other, e.g. $x = 2y$. Any rise in x also results in a rise in y - and so it is impossible to attribute any change in z to *only one of* x or y, since they *both change* at the same time.

7 Extra: RMSE of the training data

We can calculate the RMSE directly. Each model stores the residuals $\hat{y}_i - y_i$ of the training data in its `residuals` property. Squaring this, taking the mean, and taking the square root will yield the RMSE.

The R^2 of a model essentially encapsulates this training RMSE inversely, i.e. $R^2 \propto \frac{1}{\text{RMSE}}$.

```
rmse.train.slr <- sqrt(mean(m.slr$residuals ** 2))
rmse.train.quadratic <- sqrt(mean(m.quadratic$residuals ** 2))
rmse.train.loglstat <- sqrt(mean(m.loglstat$residuals ** 2))
cbind(rmse.train.slr, rmse.train.quadratic, rmse.train.loglstat)
```

```
##      rmse.train.slr rmse.train.quadratic rmse.train.loglstat
## [1,]       4.275035             3.388163            4.941973
```

8 Extra: comparing the models in the tutorial notes

The next two extra sections detail two ways to 'correct' the tutorial notes after removing outliers, so that we can compare between the models. As mentioned before, the issue in the tutorial notes is that the models use **completely different datasets**.

The first way to make the models comparable is to re-train the original model with the new training data, so that we can compare it to the new model.

```
# remove outlier data, following the tutorial notes
Boston1 <- Boston[-c(369, 373, 413),]
set.seed(100)
train.idx.outliers <- sample(nrow(Boston1), nrow(Boston1)*0.8)
train.data.outliers <- Boston1[train.idx.outliers,]
test.data.outliers <- Boston1[-train.idx.outliers,]
```

```r
# re-train original model with new training data
m.slr <- lm(medv ~ ., train.data.outliers)
rmse.outliers.slr <- RMSE(predict(m.slr, test.data.outliers),
↪  test.data.outliers$medv)

# train improved quadratic modeol with new training data
m.quadratic <- lm(medv ~ . + I(rm^2) + I(ptratio^2) + I(lstat^2),
↪  train.data.outliers)
rmse.outliers.quadratic <- RMSE(predict(m.quadratic, test.data.outliers),
↪  test.data.outliers$medv)

cbind(rmse.outliers.slr, rmse.outliers.quadratic)
```

```
##      rmse.outliers.slr rmse.outliers.quadratic
## [1,]          4.814686                3.677604
```

9 Extra: removing outliers from the training data

The second way to make the models comparable is to remove the outliers directly from the training data. In the original linear model residual plot, we had 3 outliers, with indices 369, 373, and 413. These indices are the indices from the original `Boston` dataset, and removing them directly with row indexing from `train.data` will not work.

Instead, we must remove the rows based on their row names - i.e. removing the rows with row names 369, 373, and 413. We can access a vector of the row names in a data frame using `rownames()`.

The test RMSE results of all the models are summarised in the final table. We can see that removing the outliers does improve the model based on the test RMSE.

```r
# incorrect - you can see what rows are represented by these indices
train.data1 <- train.data[-c(369, 373, 413),]
# row index 413 is not defined, since train.data only has 404 rows
# train.data[c(369, 373, 413),]

# correct - remove rows based on the row names
# rownames(train.data) %in% c(369, 373, 413) gives us the entries that are
↪  outliers.
# inverting this with the ! operator gives us all the entries that are not
↪  outliers.
train.data2 <- train.data[!(rownames(train.data) %in% c(369, 373, 413)),]

paste("Original rows in training data:", nrow(train.data))
paste("Rows in training data after wrong removal:", nrow(train.data1))
paste("Rows in training data after correct removal:", nrow(train.data2))

# the training and testing proceed as usual
```

```r
# re-train the slr model without outliers for comparison
m.slr.outliers <- lm(medv ~ ., train.data2)
rmse.outliers.slr <- RMSE(predict(m.slr.outliers, test.data),
↪  test.data$medv)

# train the quadratic model without outliers
m.quadratic.outliers <- lm(medv ~ . + I(rm^2) + I(ptratio^2) + I(lstat^2),
↪  train.data2)
rmse.outliers.quadratic <- RMSE(predict(m.quadratic.outliers, test.data),
↪  test.data$medv)

data.frame(with.outliers=c(rmse.slr, rmse.quadratic),
           rm.outliers=c(rmse.outliers.slr, rmse.outliers.quadratic),
           row.names = c("slr", "quadratic"))
```

```
## [1] "Original rows in training data: 404"
## [1] "Rows in training data after wrong removal: 402"
## [1] "Rows in training data after correct removal: 401"
## # A tibble: 2 x 2
##   with.outliers rm.outliers
##           <dbl>       <dbl>
## 1          6.28        6.19
## 2          5.80        5.59
```